

Roadmap hacia un puesto NOC

Semana 1: Fundamentos de redes y primer proyecto

Base teórica, plan diario y guía de GitHub

Martes 6 – Lunes 12 de octubre de 2026

Índice

1. Presentación de la semana	4
1.1. Objetivo	4
1.2. Cómo usar este documento	4
1.3. Carga horaria	4
1.4. Calendario resumido	4
2. Fundamentos de redes	5
2.1. Qué es una red	5
2.2. Alcance: LAN, WAN e Internet	5
2.3. Elementos básicos	5
2.4. Medios de transmisión	6
2.5. Bits, bytes y velocidad	6
3. Modelos de capas	6
3.1. Por qué existen las capas	6
3.2. El modelo TCP/IP	6
3.3. El modelo OSI	6
3.4. Encapsulación	7
4. Direccionamiento	7
4.1. Dirección MAC	7
4.2. Dirección IP	7
4.3. ARP	8
4.4. Máscara de red y notación CIDR	8
4.5. Red, broadcast y hosts	8

4.6. Método rápido por tamaño de bloque	9
4.7. Direcciones privadas	9
4.8. Gateway y tabla de rutas	9
5. Transporte, puertos y servicios	10
5.1. Puertos y sockets	10
5.2. TCP	10
5.3. UDP	10
5.4. ICMP y ping	10
5.5. TTL y traceroute	11
5.6. DNS	11
6. Herramientas de diagnóstico en Linux	12
6.1. Resumen	12
6.2. ip addr	12
6.3. ip neigh	12
6.4. dig	13
6.5. ss	13
6.6. Método de diagnóstico inicial	13
7. Proyecto 01: network-diagnostics	13
7.1. Descripción	13
7.2. Estructura del repositorio	14
7.3. Requisitos	14
7.4. Formato de salida esperado	14
7.5. Conceptos de Bash necesarios	14
7.6. Esqueleto inicial	15
7.7. Plantilla de README	16
7.8. Criterios de proyecto terminado	16
8. Git y GitHub	17
8.1. Conceptos	17
8.2. Instalación y configuración (una sola vez)	17
8.3. Autenticación con clave SSH	17
8.4. Crear y publicar el repositorio	17
8.5. Flujo de trabajo habitual	18
8.6. Buenas prácticas	18

9. Plan día por día	18
9.1. Martes 6 de octubre – Arranque y setup (1 h)	18
9.2. Miércoles 7 de octubre – Capas, IP y MAC (1 h)	18
9.3. Jueves 8 de octubre – IPv4 y subnetting (1 h)	19
9.4. Viernes 9 de octubre – Herramientas de diagnóstico (1 h)	19
9.5. Sábado 10 de octubre – Construir el proyecto (2,5 h)	19
9.6. Domingo 11 de octubre – Bonus, README y publicación (1 h)	19
9.7. Lunes 12 de octubre – Repaso y cierre (30 min)	20
9.8. Checklist final	20
10.Ejercicios y autoevaluación	20
10.1. Ejercicios de subnetting	20
10.2. Preguntas de autoevaluación	20
10.3. Soluciones de subnetting	21
10.4. Pistas para las preguntas conceptuales	21
A. Glosario	21
B. Referencia rápida de comandos	22

1. Presentación de la semana

1.1. Objetivo

Al terminar esta semana deberías poder explicar con tus palabras, y verificar con comandos, qué ocurre cuando una computadora se comunica con otra. Además, vas a tener publicado en GitHub tu primer proyecto: `network-diagnostics`.

Capacidades a alcanzar:

- Distinguir LAN, WAN e Internet.
- Explicar el modelo TCP/IP y la encapsulación.
- Distinguir dirección IP de dirección MAC y explicar para qué sirve ARP.
- Calcular red, broadcast y rango de hosts para prefijos /24, /25, /26, /27, /28 y /30.
- Leer la salida de `ip addr`, `ip route`, `ip neigh`, `ping`, `traceroute`, `dig` y `ss`.
- Diferenciar un problema de DNS de un problema de conectividad IP.
- Escribir un script Bash sencillo que genere un reporte de diagnóstico.
- Versionar y publicar un proyecto con Git y GitHub.

1.2. Cómo usar este documento

Es a la vez una guía de estudio y una fuente de consulta. Las secciones 2 a 6 son la teoría. La sección 7 describe el proyecto. La sección 8 explica cómo publicarlo. La sección 9 es el calendario día por día. Al final hay autoevaluación, soluciones y glosario.

Nota 1.1. Todo el código, los nombres de archivos y los comentarios dentro de los scripts están en inglés, que es el idioma de trabajo habitual en operaciones de red.

1.3. Carga horaria

Aproximadamente 8 horas en total. Es algo más que el promedio previsto (6–8 h) porque esta primera semana incluye la instalación de herramientas. Tu tesis sigue siendo la prioridad: si un día solo podés dedicar 30 minutos, hacé la parte más corta de la tarea y recuperá el sábado.

1.4. Calendario resumido

Día	Tiempo	Foco
Martes 6	1 h	Platzi (conceptos básicos) y setup de Git/GitHub
Miércoles 7	1 h	Modelo de capas, IP y MAC
Jueves 8	1 h	IPv4 y subnetting
Viernes 9	1 h	Herramientas de diagnóstico
Sábado 10	2,5 h	Construir el proyecto
Domingo 11	1 h	Bonus, README y publicación
Lunes 12	0,5 h	Repaso y cierre

La sección 9 detalla cada día.

2. Fundamentos de redes

2.1. Qué es una red

Definición 2.1 (Red de computadoras). Conjunto de dispositivos (computadoras, teléfonos, routers, servidores) interconectados mediante un medio de transmisión que les permite intercambiar datos siguiendo reglas comunes.

Definición 2.2 (Host). Cualquier dispositivo con una dirección propia que envía o recibe datos en la red: una PC, un servidor, un teléfono.

Definición 2.3 (Protocolo). Conjunto de reglas acordadas que define cómo se formatean, envían y reciben los datos. Dos dispositivos pueden comunicarse solo si hablan el mismo protocolo.

2.2. Alcance: LAN, WAN e Internet

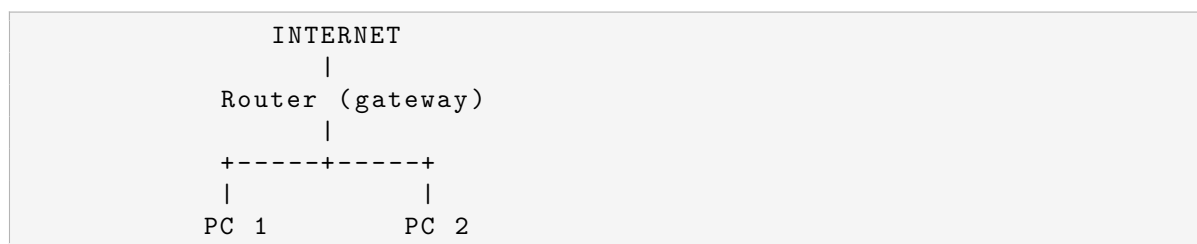
Tipo	Descripción	Ejemplo
LAN	Red local, área reducida	Tu casa, una oficina
WAN	Red de área amplia que une sitios distantes	La red de un operador entre ciudades
Internet	Red de redes interconectadas globalmente	Todo lo anterior unido

Mirada NOC 2.1. Un operador como Claro administra enormes WAN. Un NOC (*Network Operations Center*) vigila que esa red esté disponible y, cuando algo falla, lo detecta, lo diagnostica y lo escala al área correcta.

2.3. Elementos básicos

- **Switch:** conecta dispositivos dentro de una LAN.
- **Router:** conecta redes distintas entre sí y decide por dónde enviar los paquetes.
- **Gateway (puerta de enlace):** el router por el que un host sale hacia otras redes.
- **Servidor / cliente:** el servidor ofrece un servicio (web, correo, DNS) y el cliente lo consume.
- **ISP:** proveedor de acceso a Internet.

Topología mínima de referencia:



2.4. Medios de transmisión

- **Cobre** (cable de par trenzado, Ethernet): económico, distancias cortas (hasta 100 m en Ethernet estándar).
- **Fibra óptica**: transmite luz, enorme capacidad y larga distancia. Es la base de las redes troncales.
- **Inalámbrico** (Wi-Fi, radio, redes móviles): cómodo, pero compartido y más sensible a interferencias.

2.5. Bits, bytes y velocidad

Definición 2.4 (Bit y byte). Un *bit* es la unidad mínima de información (0 o 1). Un *byte* son 8 bits.

- La velocidad de transmisión se mide en *bits* por segundo: kbps, Mbps, Gbps.
- El tamaño de archivos se mide en *bytes*: KB, MB, GB.

Ejemplo 2.1. Un enlace de 100 Mbps transmite como máximo $100/8 = 12,5$ MB por segundo. Descargar un archivo de 500 MB tomaría al menos $500/12,5 = 40$ segundos.

Definición 2.5 (Latencia y ancho de banda). *Latencia*: tiempo que tarda un paquete en llegar (o en ir y volver). *Ancho de banda*: cuántos bits por segundo puede transportar el enlace. Un enlace puede tener mucho ancho de banda y alta latencia, o al revés.

3. Modelos de capas

3.1. Por qué existen las capas

Comunicar dos computadoras involucra muchos problemas distintos: señales físicas, direcciones, rutas, entrega confiable, aplicaciones. Se los separa en *capas*. Cada capa resuelve un problema y ofrece un servicio a la capa superior. Esto permite cambiar una tecnología (por ejemplo, de cable a Wi-Fi) sin rehacer todo lo demás.

3.2. El modelo TCP/IP

Capa	Función	Ejemplos
Application	Servicios que usa el usuario	HTTP, HTTPS, DNS, SSH, SMTP
Transport	Entrega entre procesos (puertos)	TCP, UDP
Internet	Direccionamiento y ruteo entre redes	IPv4, IPv6, ICMP
Network Access	Envío dentro del enlace local	Ethernet, Wi-Fi, ARP

3.3. El modelo OSI

OSI tiene siete capas. En la práctica se usa TCP/IP, pero en operaciones se habla en términos OSI (“es un problema de capa 2”). Conviene conocer la correspondencia.

N.º	OSI	Equivalencia en TCP/IP	Ejemplo
7	Aplicación	Application	HTTP, DNS
6	Presentación	Application	TLS, codificación
5	Sesión	Application	Gestión de sesiones
4	Transporte	Transport	TCP, UDP
3	Red	Internet	IP, ICMP
2	Enlace de datos	Network Access	Ethernet, MAC, ARP
1	Física	Network Access	Cobre, fibra, radio

Mirada NOC 3.1. Un orden típico de diagnóstico es de abajo hacia arriba: ¿hay enlace físico (capa 1)? ¿Hay comunicación local (capa 2)? ¿Hay IP y rutas (capa 3)? ¿Funciona el transporte (capa 4)? ¿Responde la aplicación (capa 7)?

3.4. Encapsulación

Definición 3.1 (Encapsulación). Al enviar datos, cada capa agrega su propio encabezado al mensaje que recibe de la capa superior. En el destino se hace el proceso inverso (desencapsulación).

```
Application : [ DATA ]
Transport : [TCP] [ DATA ] -> segment
Internet : [IP] [TCP] [ DATA ] -> packet
Net. Access : [ETH] [IP] [TCP] [ DATA ] [FCS] -> frame
```

Capa	Unidad de datos (PDU)
Transport (TCP)	Segmento
Transport (UDP)	Datagrama
Internet	Paquete
Network Access	Trama (<i>frame</i>)

4. Direccionamiento

4.1. Dirección MAC

Definición 4.1 (Dirección MAC). Identificador de 48 bits de una interfaz de red, escrito en hexadecimal (por ejemplo `3c:52:82:aa:10:5f`). Opera en la capa de enlace y solo tiene significado dentro de la red local.

4.2. Dirección IP

Definición 4.2 (Dirección IPv4). Número de 32 bits que identifica a un host en una red y permite el ruteo entre redes. Se escribe como cuatro números decimales (0–255) separados por puntos.

Ejemplo 4.1 (Conversión a binario). 192.168.1.20 se escribe como

$$\underbrace{11000000}_{192}.\underbrace{10101000}_{168}.\underbrace{00000001}_1.\underbrace{00010100}_{20}.$$

Definición 4.3 (IP frente a MAC). La IP indica *dónde* está el host dentro de la red y puede cambiar (por ejemplo, si te conectás a otra red). La MAC identifica la *placa* de red y se usa solo en el enlace local.

4.3. ARP

Definición 4.4 (ARP). *Address Resolution Protocol*: protocolo que, dentro de una LAN, permite averiguar la MAC asociada a una IP conocida. El host pregunta por broadcast “¿quién tiene la IP X?” y el dueño responde con su MAC.

Ejemplo 4.2 (Salir a Internet). Tu PC (192.168.1.20) quiere llegar a 8.8.8.8. Esa IP está fuera de tu red, así que el paquete se entrega al gateway (192.168.1.1). Para armar la trama Ethernet, tu PC necesita la *MAC del gateway*, que obtiene con ARP. La IP destino del paquete sigue siendo 8.8.8.8; solo la MAC destino de la trama es la del gateway.

4.4. Máscara de red y notación CIDR

Definición 4.5 (Máscara y prefijo). La máscara indica qué bits de la dirección identifican la *red* y cuáles identifican el *host*. La notación CIDR /*n* indica que los primeros *n* bits son de red.

Ejemplo 4.3. /24 equivale a la máscara 11111111.11111111.11111111.00000000 = 255.255.255.0. Los 24 primeros bits son de red y los 8 restantes de host.

4.5. Red, broadcast y hosts

Dada una dirección con prefijo /*n*:

- **Dirección de red:** todos los bits de host en 0. Identifica la red.
- **Dirección de broadcast:** todos los bits de host en 1. Llega a todos los hosts de la red.
- **Hosts útiles:** las direcciones entre ambas.

$$\text{direcciones totales} = 2^{32-n}, \quad \text{hosts útiles} = 2^{32-n} - 2.$$

Prefijo	Máscara	Direcciones	Hosts útiles
/8	255.0.0.0	16 777 216	16 777 214
/16	255.255.0.0	65 536	65 534
/24	255.255.255.0	256	254
/25	255.255.255.128	128	126
/26	255.255.255.192	64	62
/27	255.255.255.224	32	30
/28	255.255.255.240	16	14
/30	255.255.255.252	4	2

Nota 4.1. Un enlace punto a punto entre dos routers suele usar /30: dos hosts útiles, uno por extremo.

4.6. Método rápido por tamaño de bloque

Para prefijos entre /25 y /30, el cambio ocurre en el cuarto octeto:

$$\text{tamaño de bloque} = 256 - (\text{cuarto octeto de la máscara}).$$

Las redes empiezan en múltiplos del tamaño de bloque. La dirección de red es el múltiplo más grande que no supera el último octeto del host. El broadcast es la dirección de red más el tamaño de bloque menos 1.

Ejemplo 4.4 (Resuelto: 192.168.1.70/26). La máscara es 255.255.255.192, así que el bloque es $256 - 192 = 64$. Las redes comienzan en 0, 64, 128, 192. Como $64 \leq 70 < 128$:

- Red: 192.168.1.64
- Broadcast: $64 + 64 - 1 = 127$, es decir 192.168.1.127
- Hosts útiles: de 192.168.1.65 a 192.168.1.126 (62 hosts)

Verificación en binario: $70 = 01000110$; con la máscara 11000000 queda $01000000 = 64$.

4.7. Direcciones privadas

Rango	Uso
10.0.0.0/8	Privada (redes grandes)
172.16.0.0/12	Privada
192.168.0.0/16	Privada (hogares y oficinas)
127.0.0.0/8	Loopback (la propia máquina)

Las direcciones privadas no se rutean en Internet. Para salir se usa NAT, que veremos más adelante en el roadmap.

4.8. Gateway y tabla de rutas

Definición 4.6 (Tabla de rutas). Lista de reglas que indica por qué interfaz y a través de qué siguiente salto se envía un paquete según su destino.

```
default via 192.168.1.1 dev wlp2s0 proto dhcp metric 600
192.168.1.0/24 dev wlp2s0 proto kernel scope link src 192.168.1.20
```

Interpretación:

- Primera línea: todo destino que no esté en una ruta más específica (*default*) se envía al gateway 192.168.1.1.
- Segunda línea: la red 192.168.1.0/24 es local; se alcanza directamente por la interfaz wlp2s0, con origen 192.168.1.20.

Mirada NOC 4.1. Si el host no tiene ruta por defecto o el gateway no responde, no hay salida a Internet aunque la red local funcione. Es de los primeros puntos a revisar.

5. Transporte, puertos y servicios

5.1. Puertos y sockets

Definición 5.1 (Puerto). Número de 16 bits (0–65535) que identifica un servicio o proceso dentro de un host. La IP dice a qué host; el puerto dice a qué servicio.

Definición 5.2 (Socket). Combinación de dirección IP y puerto, por ejemplo 192.168.1.20:22.

Puerto	Servicio	Transporte habitual
22	SSH	TCP
25	SMTP (correo)	TCP
53	DNS	UDP (y TCP)
67/68	DHCP	UDP
80	HTTP	TCP
443	HTTPS	TCP

5.2. TCP

Definición 5.3 (TCP). Protocolo de transporte *orientado a conexión* y confiable: establece una conexión, numera los datos, confirma la recepción y retransmite lo perdido.

El inicio de la conexión es el *saludo de tres vías* (*three-way handshake*):

Client	Server
----- SYN ----->	
<----- SYN-ACK -----	
----- ACK ----->	
connection open	

5.3. UDP

Definición 5.4 (UDP). Protocolo de transporte *sin conexión*: envía datagramas sin handshake ni confirmación. Tiene menos sobrecarga, pero no garantiza entrega ni orden.

	TCP	UDP
Conexión	Sí (handshake)	No
Fiabilidad	Confirmaciones y retransmisión	Ninguna
Sobrecarga	Mayor	Menor
Usos típicos	Web, SSH, correo	DNS, streaming, voz

5.4. ICMP y ping

Definición 5.5 (ICMP). *Internet Control Message Protocol*: protocolo de la capa Internet para mensajes de control y diagnóstico. **ping** usa *Echo Request* y espera un *Echo Reply*.

```
$ ping -c 4 8.8.8.8
64 bytes from 8.8.8.8: icmp_seq=1 ttl=118 time=20.1 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=118 time=19.7 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=118 time=21.3 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=118 time=20.5 ms

--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 19.7/20.4/21.3/0.6 ms
```

(Salida ilustrativa: los valores reales variarán.)

- **time:** latencia de ida y vuelta (RTT) en milisegundos.
- **packet loss:** porcentaje de paquetes sin respuesta.
- **ttl:** tiempo de vida restante del paquete.

Mirada NOC 5.1. “Pérdida de paquetes” y “aumento de latencia” son dos síntomas que vas a escribir y leer a diario. Un ping que falla no prueba por sí solo que el destino esté caído: algunos equipos bloquean ICMP a propósito.

5.5. TTL y traceroute

Definición 5.6 (TTL). *Time To Live*: campo del encabezado IP que cada router decrementa en 1. Si llega a 0, el paquete se descarta y el router avisa con un mensaje ICMP.

traceroute aprovecha esto: envía paquetes con TTL 1, 2, 3, ... y registra qué router responde en cada salto, revelando el camino.

```
$ traceroute google.com
1  192.168.1.1      1.2 ms
2  10.20.0.1       8.5 ms
3  200.xx.xx.xx    12.1 ms
4  * * *
5  142.xxx.xxx.xxx 21.0 ms
```

Una línea con * * * significa que ese salto no respondió. No siempre indica un fallo: muchos routers no contestan estos mensajes pero igualmente reenvían el tráfico.

5.6. DNS

Definición 5.7 (DNS). *Domain Name System*: sistema distribuido que traduce nombres (**google.com**) a direcciones IP.

Flujo simplificado de una consulta:

```
PC -> resolver (e.g. router or ISP) -> root servers
    -> TLD servers (.com) -> authoritative server -> IP address
```

El resolver guarda en caché las respuestas durante un tiempo (TTL del registro).

Registro	Contenido
A	Nombre → dirección IPv4
AAAA	Nombre → dirección IPv6
CNAME	Alias de otro nombre
MX	Servidor de correo del dominio
NS	Servidores de nombres autoritativos

Ejemplo 5.1 (Separar DNS de conectividad IP). Si `ping 8.8.8.8` funciona pero `ping google.com` falla con “Name or service not known”, hay conectividad IP pero la resolución de nombres no funciona. El problema está en DNS, no en la red.

6. Herramientas de diagnóstico en Linux

6.1. Resumen

Comando	Para qué sirve
<code>hostname</code>	Nombre de la máquina
<code>ip addr</code>	Interfaces, direcciones IP y MAC
<code>ip route</code>	Tabla de rutas y gateway
<code>ip neigh</code>	Tabla de vecinos (ARP)
<code>ping</code>	Conectividad y latencia (ICMP)
<code>traceroute</code>	Camino hacia un destino
<code>dig / nslookup</code>	Consultas DNS
<code>ss -tuln</code>	Puertos TCP/UDP en escucha

6.2. `ip addr`

```
$ ip addr
2: wlp2s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 ...
    link/ether 3c:52:82:aa:10:5f brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.20/24 brd 192.168.1.255 scope global dynamic wlp2s0
```

- `wlp2s0`: nombre de la interfaz.
- `UP,LOWER_UP`: interfaz activa y con enlace físico.
- `link/ether`: dirección MAC.
- `inet 192.168.1.20/24`: dirección IPv4 y prefijo.

6.3. `ip neigh`

```
$ ip neigh
192.168.1.1 dev wlp2s0 lladdr 00:11:22:33:44:55 REACHABLE
```

Muestra la MAC (`lladdr`) asociada a cada IP vecina, aprendida por ARP. El estado `REACHABLE` indica que se confirmó recientemente; `STALE` indica que la entrada es antigua; `FAILED` indica que no hubo respuesta.

6.4. dig

```
$ dig google.com +short
142.250.xxx.xxx
```

Sin `+short` se ve la respuesta completa: la sección **ANSWER**, el TTL del registro y el servidor que respondió. Para consultar otro tipo de registro: `dig google.com MX`.

6.5. ss

```
$ ss -tuln
Netid State  Local Address:Port
tcp    LISTEN  0.0.0.0:22
udp    UNCONN  127.0.0.53:53
```

Las opciones: `-t` TCP, `-u` UDP, `-l` en escucha (*listening*), `-n` muestra números en lugar de nombres. `0.0.0.0:22` significa que hay un servicio escuchando en el puerto 22 en todas las interfaces.

6.6. Método de diagnóstico inicial

Ante “no tengo Internet”, no se tocan configuraciones al azar. Se sigue una cadena de verificaciones, de lo más cercano a lo más lejano:

Paso	Pregunta	Herramienta
1	¿La interfaz está activa y tiene IP?	<code>ip addr</code>
2	¿Hay ruta por defecto?	<code>ip route</code>
3	¿Responde el gateway?	ping a la IP del gateway
4	¿Hay salida a Internet por IP?	<code>ping 8.8.8.8</code>
5	¿Funciona DNS?	<code>dig google.com</code>
6	¿Dónde se corta el camino?	<code>traceroute</code>

Síntoma	Causa probable
Sin IP en la interfaz	Problema local o de DHCP
Responde el gateway, falla <code>8.8.8.8</code>	Problema más allá de la LAN (router, ISP)
Responde <code>8.8.8.8</code> , falla el nombre	Problema de DNS
Latencia alta y pérdida intermitente	Congestión o enlace degradado

7. Proyecto 01: network-diagnostics

7.1. Descripción

Un script Bash que inspecciona la configuración de red de una máquina Linux, ejecuta pruebas de conectividad y genera un reporte legible con resultados `[OK]` o `[FAIL]`.

7.2. Estructura del repositorio

```
network-diagnostics/
|-- README.md
|-- network_diagnostics.sh
|-- results/
|   '-- example.txt
'-- docs/
    '-- week01.md
```

7.3. Requisitos

1. Mostrar el hostname y la configuración IP (`ip addr`).
2. Mostrar la tabla de rutas (`ip route`) y la tabla de vecinos (`ip neigh`).
3. Probar la conectividad a Internet con `ping` a 8.8.8.8 e imprimir [OK] o [FAIL].
4. Probar la resolución DNS de `google.com` e imprimir [OK] o [FAIL].
5. Listar los puertos en escucha (`ss -tuln`).
6. Código y comentarios en inglés.

Bonus: aceptar un host como argumento (`./network_diagnostics.sh google.com`) y mostrar: resolución DNS, IP obtenida, ping OK/FAIL, latencia promedio y porcentaje de pérdida.

7.4. Formato de salida esperado

```
=====
NETWORK DIAGNOSTICS REPORT
=====

HOSTNAME
-----
my-laptop

INTERNET CONNECTIVITY
-----
[OK] Internet connectivity

DNS
-----
[OK] DNS resolution
```

7.5. Conceptos de Bash necesarios

Definición 7.1 (Código de salida). Todo comando termina con un número: 0 significa éxito y cualquier otro valor indica error. La variable `$?` guarda el código del último comando. Es la base de cualquier chequeo automático.

```
# 1. Shebang: the first line selects the interpreter
#!/usr/bin/env bash

# 2. Functions and return codes
check_internet() {
    ping -c 1 -W 2 8.8.8.8 > /dev/null 2>&1
}

# 3. Conditionals using the exit code
if check_internet; then
    echo "[OK] Internet connectivity"
else
    echo "[FAIL] Internet connectivity"
fi

# 4. Positional argument with a default value
TARGET="${1:-google.com}"
```

- `> /dev/null 2>&1` descarta la salida normal y la de errores.
- `${1:-valor}` usa el primer argumento, o valor si no se pasó.
- `$(comando)` captura la salida de un comando en una variable.

7.6. Esqueleto inicial

Completá las partes marcadas con TODO.

```
#!/usr/bin/env bash

section() {
    echo
    echo "$1"
    echo "-----"
}

check_internet() {
    # TODO: ping 8.8.8.8 once and return success or failure
    :
}

check_dns() {
    # TODO: resolve google.com and return success or failure
    :
}

echo "===== "
echo "NETWORK DIAGNOSTICS REPORT"
echo "===== "

section "HOSTNAME"
hostname

section "IP CONFIGURATION"
# TODO: ip addr
```

```
# TODO: routing table, neighbor table, DNS check, listening ports
```

Ejecución:

```
chmod +x network_diagnostics.sh
./network_diagnostics.sh
./network_diagnostics.sh > results/example.txt
```

Nota 7.1. Antes de publicar `results/example.txt`, revisá que no incluya información que no quieras hacer pública.

7.7. Plantilla de README

Un buen README explica qué hace el proyecto, cómo usarlo y qué se aprendió.

```
# Network Diagnostics

A Bash script that checks the network status of a Linux machine
and prints a readable report.

## What it checks
- IP configuration, routing table and ARP/neighbor table
- Internet connectivity (ICMP)
- DNS resolution
- Listening ports

## Usage
    chmod +x network_diagnostics.sh
    ./network_diagnostics.sh
    ./network_diagnostics.sh example.com

## Example output
(paste a trimmed sample here)

## What I learned
- IP vs MAC, gateways and routing tables
- How to tell a DNS problem from a connectivity problem
- Basic Bash scripting for diagnostics

## Next steps
- Add subnet calculations
- Export results to a log file
```

7.8. Criterios de proyecto terminado

- ☐ El script corre sin errores en tu máquina.
- ☐ Cada prueba muestra [OK] o [FAIL].
- ☐ Hay un ejemplo real en `results/`.
- ☐ El README es claro y se ve bien renderizado en GitHub.
- ☐ Las notas de `docs/week01.md` están completas.

8. Git y GitHub

8.1. Conceptos

Definición 8.1 (Git). Sistema de control de versiones que guarda el historial de cambios de tus archivos.

Definición 8.2 (GitHub). Servicio web que aloja repositorios Git. Es tu vidriera profesional: quienes revisen tu perfil verán tus proyectos.

- **Repositorio:** carpeta con el proyecto y su historial.
- **Commit:** captura de un estado del proyecto, con un mensaje descriptivo.
- **Remote (origin):** copia del repositorio en GitHub.
- **Push:** enviar commits locales a GitHub.
- **Branch (main):** línea de desarrollo principal.

8.2. Instalación y configuración (una sola vez)

```
sudo apt install git
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

8.3. Autenticación con clave SSH

```
ssh-keygen -t ed25519 -C "you@example.com"
cat ~/.ssh/id_ed25519.pub
```

Copía la salida del segundo comando y pégala en GitHub: *Settings* → *SSH and GPG keys* → *New SSH key*. Verifícalo con:

```
ssh -T git@github.com
```

8.4. Crear y publicar el repositorio

1. En github.com, botón *New*. Nombre: **network-diagnostics**. Público, **sin** README ni **.gitignore** (los creas localmente).
2. En tu máquina:

```
cd network-diagnostics
git init
git add .
git commit -m "Add initial project structure"
git branch -M main
git remote add origin git@github.com:YOUR_USERNAME/network-diagnostics
git push -u origin main
```

8.5. Flujo de trabajo habitual

```
git status          # what changed
git add .           # stage changes
git commit -m "Add DNS check" # save a snapshot
git push            # upload to GitHub
git log --oneline   # view history
```

8.6. Buenas prácticas

- Commits chicos y frecuentes, con mensajes en inglés y en imperativo (“Add...”, “Fix...”).
- Nunca subas contraseñas, tokens ni datos personales.
- Un repositorio por proyecto semanal. Más adelante armaremos un README de perfil que los enlace.

9. Plan día por día

9.1. Martes 6 de octubre – Arranque y setup (1 h)

Estudiar: sección 2.

- ☐ Platzi: Curso de Fundamentos de Redes, lecciones 2 a 6 (aprox. 24 min en total). Las lecciones siguientes están bloqueadas y probablemente se habilitan en orden.
- ☐ Tomar notas con tus palabras (LAN, WAN, medios, bits y bytes, importancia de las redes).
- ☐ Crear tu cuenta de GitHub, si todavía no la tenés.
- ☐ Instalar y configurar Git (sección 8).
- ☐ Crear la carpeta `network-diagnostics/` con la estructura de la sección 7.
- ☐ Crear el repositorio vacío en GitHub.

Entregable: repositorio creado.

9.2. Miércoles 7 de octubre – Capas, IP y MAC (1 h)

Estudiar: secciones 3 y 4 (hasta ARP).

- ☐ Platzi: lecciones “Protocolos y Capas” y “Modelo TCP/IP”.
- ☐ En `docs/week01.md`, resumir las cuatro capas de TCP/IP con tus palabras.
- ☐ Ejecutar `ip addr` y `ip neigh`. Anotar tu IP, tu MAC y la MAC de tu gateway.
- ☐ Dibujar a mano el viaje de un paquete desde tu PC a 8.8.8.8, indicando qué IP y qué MAC se usan en cada tramo.

Pregunta guía: ¿por qué tu PC necesita la MAC del gateway para salir a Internet?

9.3. Jueves 8 de octubre – IPv4 y subnetting (1 h)

Estudiar: resto de la sección 4.

- ☐ Platzi: lección “Direcciones IP”.
- ☐ Memorizar la tabla de prefijos (/24, /25, /26, /27, /28, /30).
- ☐ Resolver los ejercicios de la sección 10.1 sin mirar las soluciones.
- ☐ Ejecutar `ip route` e interpretar cada línea en `docs/week01.md`.

9.4. Viernes 9 de octubre – Herramientas de diagnóstico (1 h)

Estudiar: secciones 5 y 6.

- ☐ Ejecutar y anotar qué significa cada salida:

```
ping -c 4 8.8.8.8
ping -c 4 google.com
traceroute google.com      # if missing: sudo apt install traceroute
dig google.com
ss -tuln
```

- ☐ Identificar en la salida de `ss` al menos un servicio en escucha y su puerto.
- ☐ Memorizar los puertos: 22, 25, 53, 67/68, 80, 443.
- ☐ Practicar el método de diagnóstico de la sección 6.

9.5. Sábado 10 de octubre – Construir el proyecto (2,5 h)

Estudiar: sección 7.

- ☐ Escribir `network_diagnostics.sh` con los requisitos de la sección 7.
- ☐ Probarlo con conexión y también sin conexión (por ejemplo, desactivando el Wi-Fi) para ver los [FAIL].
- ☐ Guardar una salida en `results/example.txt`.
- ☐ Hacer un primer commit con el script funcionando.

9.6. Domingo 11 de octubre – Bonus, README y publicación (1 h)

- ☐ Bonus: aceptar un host como argumento y mostrar DNS, IP, ping, latencia promedio y pérdida.
- ☐ Escribir el `README.md` con la plantilla de la sección 7.
- ☐ Completar `docs/week01.md`.
- ☐ `git add .`, `git commit` y `git push`.
- ☐ Verificar en GitHub que el README se vea bien.

9.7. Lunes 12 de octubre – Repaso y cierre (30 min)

- ☐ Responder de memoria la autoevaluación (sección 10).
- ☐ Rehacer los ejercicios de subnetting que más te costaron.
- ☐ Revisar el repositorio como si fueras un reclutador: ¿se entiende qué hace en 30 segundos?
- ☐ Anotar tus dudas y compartirlas para preparar la Semana 2 (Linux básico y switching).

9.8. Checklist final

- ☐ Platzi: lecciones de Conceptos básicos completadas.
- ☐ Notas en `docs/week01.md`.
- ☐ Seis ejercicios de subnetting resueltos.
- ☐ Script con reporte y chequeos [OK]/[FAIL].
- ☐ Bonus con host como argumento.
- ☐ README completo.
- ☐ Repositorio público en GitHub.

10. Ejercicios y autoevaluación

10.1. Ejercicios de subnetting

Para cada dirección, hallá la dirección de red, el broadcast y el rango de hosts útiles.

1. 192.168.10.37/24
2. 10.0.5.200/25
3. 192.168.1.70/26
4. 172.16.4.100/27
5. 192.168.0.45/28
6. 192.168.5.9/30

10.2. Preguntas de autoevaluación

1. ¿Qué ocurre, paso a paso, cuando ejecutás `ping 8.8.8.8`?
2. ¿Qué diferencia hay entre una dirección IP y una MAC?
3. ¿Qué es un gateway y en qué comando lo ves?
4. ¿Cómo distinguís un fallo de DNS de un fallo de conectividad IP?
5. ¿Qué hace TCP que UDP no hace?

6. ¿Para qué sirve ARP y en qué capa actúa?
7. ¿Qué significa un * * * en `traceroute`?
8. ¿Cuántos hosts útiles tiene una red /27? ¿Y una /30?
9. ¿Qué indica el código de salida 0 en un comando?
10. Un colega dice “no hay Internet”. ¿Cuáles son tus primeras cuatro verificaciones y en qué orden?

10.3. Soluciones de subnetting

N.º	Red	Broadcast	Hosts útiles
1	192.168.10.0	192.168.10.255	.1 a .254
2	10.0.5.128	10.0.5.255	.129 a .254
3	192.168.1.64	192.168.1.127	.65 a .126
4	172.16.4.96	172.16.4.127	.97 a .126
5	192.168.0.32	192.168.0.47	.33 a .46
6	192.168.5.8	192.168.5.11	.9 a .10

10.4. Pistas para las preguntas conceptuales

- **Pregunta 1:** hay resolución de nombre (si usás nombre), ARP para obtener la MAC del gateway, un paquete IP con ICMP Echo Request, el reenvío por los routers y un Echo Reply de vuelta.
- **Pregunta 4:** si funciona `ping` por IP pero no por nombre, el problema es DNS.
- **Pregunta 8:** /27: $2^5 - 2 = 30$; /30: $2^2 - 2 = 2$.
- **Pregunta 10:** `ip addr`, `ip route`, `ping` al gateway, `ping` a una IP pública.

A. Glosario

ARP

Protocolo que resuelve una IP a una MAC dentro de la LAN.

Broadcast

Dirección que alcanza a todos los hosts de una red.

CIDR

Notación /n que indica cuántos bits de la dirección corresponden a la red.

DNS

Sistema que traduce nombres de dominio a direcciones IP.

Encapsulación

Agregado de encabezados por cada capa al enviar datos.

Gateway

Router por el que un host sale hacia otras redes.

Host

Dispositivo con dirección propia en una red.

ICMP

Protocolo de control y diagnóstico de la capa Internet; usado por **ping**.

Latencia

Tiempo que tarda un paquete en llegar o en ir y volver.

LAN / WAN

Red local / red de área amplia.

MAC

Dirección de 48 bits de una interfaz de red.

NOC

Network Operations Center: centro que monitorea y gestiona la red.

Packet loss

Porcentaje de paquetes que no llegan.

Puerto

Número que identifica un servicio dentro de un host.

Router

Dispositivo que reenvía paquetes entre redes distintas.

Socket

Par IP y puerto.

Switch

Dispositivo que conecta hosts dentro de una LAN.

TCP / UDP

Protocolos de transporte: con conexión y confiable / sin conexión.

TTL

Contador del paquete IP que limita el número de saltos.

B. Referencia rápida de comandos

```
# Network configuration
ip addr                # interfaces, IPs, MACs
ip route               # routing table
ip neigh               # ARP table

# Connectivity
ping -c 4 8.8.8.8      # ICMP test, 4 packets
traceroute google.com  # path to a destination

# DNS
dig google.com         # full DNS answer
dig google.com +short  # only the answer
nslookup google.com    # alternative DNS query

# Sockets
```

```
ss -tuln                                # listening TCP/UDP ports

# Git
git status
git add .
git commit -m "Message"
git push
git log --oneline
```